Course Title: Cryptography and Network Security Lab	**Course Code**: BCSE309P
Faculty: Prof. Karthika V	Slot: L15+L16
Regno: *23BCE1344*	Name: *Aditya*

EX1B

PlayFair and Hill Cipher

Index	
Crypto_function.py	Driver Function - Encryption/Decryption Algorithms
Code (Sender, Receiver)	Server/Client .py codes (need minimal changes in code to execute)
Output (Sender, Receiver)	Screenshot of server-client communication.

File name: Ex1b _23BCE1344.pdf

Code: (--CyptoFunctions--)

Cypto_functions.py:

```
import string

def generate_matrix(key):
    key = key.upper().replace("J", "I")
    matrix = []
    used = set()
    for ch in key + string.ascii_uppercase:
        if ch == "J":
            continue
        if ch not in used:
            used.add(ch)
            matrix.append(ch)

    return [matrix[i:i+5] for i in range(0, 25, 5)]

def prepare_text(text):
    text = text.upper().replace("J", "I")
    text = "".join(ch for ch in text if ch.isalpha())
    result = ""
    i = 0
    while i < len(text):
        a = text[i]
        b = text[i+1] if i+1 < len(text) else "X"
        if a == b:
            result += a + "X"
            i += 1
        else:
            result += a + b
            i += 2
    if len(result) % 2 != 0:
        result += "X"
    return result

def find_pos(matrix, ch):
    for i in range(5):
        for j in range(5):
            if matrix[i][j] == ch:
                return i, j

def playfair_encrypt(text, key):
    matrix = generate_matrix(key)
    text = prepare_text(text)
    cipher = ""
    for i in range(0, len(text), 2):
        a, b = text[i], text[i+1]
        r1, c1 = find_pos(matrix, a)
        r2, c2 = find_pos(matrix, b)
        if r1 == r2:
            cipher += matrix[r1][(c1+1)%5] + matrix[r2][(c2+1)%5]
```

```
        elif c1 == c2:
            cipher += matrix[(r1+1)%5][c1] + matrix[(r2+1)%5][c2]
        else:
            cipher += matrix[r1][c2] + matrix[r2][c1]
    return cipher

def playfair_decrypt(text, key):
    matrix = generate_matrix(key)
    plain = ""
    for i in range(0, len(text), 2):
        a, b = text[i], text[i+1]
        r1, c1 = find_pos(matrix, a)
        r2, c2 = find_pos(matrix, b)
        if r1 == r2:
            plain += matrix[r1][(c1-1)%5] + matrix[r2][(c2-1)%5]
        elif c1 == c2:
            plain += matrix[(r1-1)%5][c1] + matrix[(r2-1)%5][c2]
        else:
            plain += matrix[r1][c2] + matrix[r2][c1]
    return plain



def hill_encrypt(text, key):
    text = text.upper().replace(" ", "")
    if len(text) % 2 != 0:
        text += "X"
    cipher = ""
    for i in range(0, len(text), 2):
        p1 = ord(text[i]) - ord('A')
        p2 = ord(text[i+1]) - ord('A')
        c1 = (key[0][0]*p1 + key[0][1]*p2) % 26
        c2 = (key[1][0]*p1 + key[1][1]*p2) % 26
        cipher += chr(c1 + ord('A')) + chr(c2 + ord('A'))
    return cipher


def hill_decrypt(text, inv_key):
    plain = ""
    for i in range(0, len(text), 2):
        c1 = ord(text[i]) - ord('A')
        c2 = ord(text[i+1]) - ord('A')
        p1 = (inv_key[0][0]*c1 + inv_key[0][1]*c2) % 26
        p2 = (inv_key[1][0]*c1 + inv_key[1][1]*c2) % 26
        plain += chr(p1 + ord('A')) + chr(p2 + ord('A'))
    return plain


def main():
    print("----------- PLAYFAIR CIPHER TEST -----------")
    playfair_key = "MONARCHY"
    message = "HELLO WORLD"
    print("Original Message       :", message)
```

```python
    print("Key Used                :", playfair_key)
    prepared_text = prepare_text(message)
    print("Prepared Plaintext      :", prepared_text)
    matrix = generate_matrix(playfair_key)
    print("Playfair Key Matrix     :")
    for row in matrix:
        print(row)
    encrypted = playfair_encrypt(message, playfair_key)
    print("Encrypted Message      :", encrypted)
    decrypted = playfair_decrypt(encrypted, playfair_key)
    print("Decrypted Message      :", decrypted)
    print("\n")

    print("---------- HILL CIPHER TEST ----------")
    hill_key = [[3, 3],[2, 5]]
    hill_inv_key = [[15, 17],[20, 9]]
    message = "HELLO"
    print("Original Message        :", message)
    print("Key Matrix              :", hill_key)
    print("Inverse Key Matrix      :", hill_inv_key)
    if len(message) % 2 != 0:
        print("Padding applied         : X")
    encrypted = hill_encrypt(message, hill_key)
    print("Encrypted Message      :", encrypted)
    decrypted = hill_decrypt(encrypted, hill_inv_key)
    print("Decrypted Message      :", decrypted)
if __name__ == "__main__":
    main()
```

Output:

```
(lab) ak@DevLab:~/Desktop/Data/College/Semester6/Assignment/Crypto Lab/Ex1b$ python3 ./crypto_functions.py
----------- PLAYFAIR CIPHER TEST -----------
Original Message        : HELLO WORLD
Key Used                : MONARCHY
Prepared Plaintext      : HELXLOWORLDX
Playfair Key Matrix     :
['M', 'O', 'N', 'A', 'R']
['C', 'H', 'Y', 'B', 'D']
['E', 'F', 'G', 'I', 'K']
['L', 'P', 'Q', 'S', 'T']
['U', 'V', 'W', 'X', 'Z']
Encrypted Message      : CFSUPMVNMTBZ
Decrypted Message      : HELXLOWORLDX


---------- HILL CIPHER TEST ----------
Original Message        : HELLO
Key Matrix              : [[3, 3], [2, 5]]
Inverse Key Matrix      : [[15, 17], [20, 9]]
Padding applied         : X
Encrypted Message      : HIOZHN
Decrypted Message      : HELLOX
```

Code: (--Playfair Cipher--)

Sender (server.py):

```
import socket
import crypto_functions as cf

playfair_key = "MONARCHY"
hillcipher_key = [[3, 3],[2, 5]]
hillcipher_inv_key = [[15, 17],[20, 9]]

def encrypt(text):
   return cf.playfair_encrypt(text, playfair_key)
def decrypt(text):
   return cf.playfair_decrypt(text, playfair_key)

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
host = "127.0.0.1"
port = 12345
server_socket.bind((host, port))
server_socket.listen(1)

print(f"Server listening on {host}:{port}")
connection, address = server_socket.accept()
print(f"Connected to {address}")

while True:
   data = connection.recv(1024).decode()
   print(f"Encrypted message from client: {data}")
   data = decrypt(data)
   print(f"Decrypted message from client: {data}\n")
   if data.lower() == "exit":
       print("Exiting...")
       break
   msg = input("You: ")
   enc_msg = encrypt(msg)
   print(f"Encrypted message sent to client: {enc_msg}\n")
   connection.send(enc_msg.encode())

connection.close()
server_socket.close()
```

Receiver (client.py):

```
iimport socket
import crypto_functions as cf

playfair_key = "MONARCHY"
hillcipher_key = [[3, 3],[2, 5]]
hillcipher_inv_key = [[15, 17],[20, 9]]

def encrypt(text):
   return cf.playfair_encrypt(text, playfair_key)
def decrypt(text):
   return cf.playfair_decrypt(text, playfair_key)

client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
host = "127.0.0.1"
port = 12345
client_socket.connect((host, port))
print("Connected to the server")

while True:
   message = input("You: ")
   message = encrypt(message)
   print(f"Encrypted message sent to server: {message}\n")
   client_socket.send(message.encode())
   if decrypt(message).lower() == "exit":
       break
   msg = client_socket.recv(1024).decode()
   print(f"Encrypted message from server: {msg}")
   data = decrypt(msg)
   print(f"Decrypted message from server: {data}\n")

client_socket.close()
```

Output:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

(lab) ak@DevLab:~/Desktop/Data/College/Semester6/Assignment/Cryp
to Lab/Ex1b$ python3 ./server.py
Server listening on 127.0.0.1:12345
Connected to ('127.0.0.1', 47388)
Encrypted message from client: CFSUPMPDKMFCNVRMGCMV
Decrypted message from client: HELXLOTHEREHOWAREYOU

You: Great, how are you?
Encrypted message sent to client: KNIMPDNVRMGCMV

Encrypted message from client: GKMGIKUFMBELXATLNOSXXSULHMCKANXZ
Decrypted message from client: FINEGIVEACCESXSTOMISSILECODENOWX

You: GHYXZ iiiiis thee codeexx
Encrypted message sent to client: FYBWXKSASASASXPDIULERHIUIUZZ

Encrypted message from client: DBIUIUKS
Decrypted message from client: BYEXEXIT

You: ok
Encrypted message sent to client: RF

Encrypted message from client: IUKS
Decrypted message from client: EXIT

Exiting...
(lab) ak@DevLab:~/Desktop/Data/College/Semester6/Assignment/Cryp
to Lab/Ex1b$
```

```
python3   bash   python3

(lab) ak@DevLab:~/Desktop/Data/College/Semester6/Assignment/Cryp
to Lab/Ex1b$ python3 ./client.py
Connected to the server
You: Hello There? How are you?
Encrypted message sent to server: CFSUPMPDKMFCNVRMGCMV

Encrypted message from server: KNIMPDNVRMGCMV
Decrypted message from server: GREATHOWAREYOU

You: Fine, Give Access to Missile Code now!!
Encrypted message sent to server: GKMGIKUFMBELXATLNOSXXSULHMCKAN
XZ

Encrypted message from server: FYBWXKSASASASXPDIULERHIUIUZZ
Decrypted message from server: GHYXZIIXIXIXISTHEXECODEXEXXX

You: bye exit
Encrypted message sent to server: DBIUIUKS

Encrypted message from server: RF
Decrypted message from server: OK

You: exit
Encrypted message sent to server: IUKS

Encrypted message from server:
Decrypted message from server:

You:
```

Code: (--Hill Cipher--)

Sender (server.py):

```
import socket
import crypto_functions as cf

playfair_key = "MONARCHY"
hillcipher_key = [[3, 3],[2, 5]]
hillcipher_inv_key = [[15, 17],[20, 9]]   # Precomputed inverse

def encrypt(text):
   return cf.hill_encrypt(text, hillcipher_key)
def decrypt(text):
   return cf.hill_decrypt(text, hillcipher_inv_key)

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
host = "127.0.0.1"
port = 12345
server_socket.bind((host, port))
server_socket.listen(1)

print(f"Server listening on {host}:{port}")
connection, address = server_socket.accept()
print(f"Connected to {address}")

while True:
   data = connection.recv(1024).decode()
   print(f"Encrypted message from client: {data}")
   data = decrypt(data)
   print(f"Decrypted message from client: {data}\n")
   if data.lower() == "exit":
       print("Exiting...")
       break
   msg = input("You: ")
   enc_msg = encrypt(msg)
   print(f"Encrypted message sent to client: {enc_msg}\n")
   connection.send(enc_msg.encode())

connection.close()
server_socket.close()
```

Receiver (client.py):

```python
import socket
import crypto_functions as cf

playfair_key = "MONARCHY"
hillcipher_key = [[3, 3],[2, 5]]
hillcipher_inv_key = [[15, 17],[20, 9]]

def encrypt(text):
   return cf.hill_encrypt(text, hillcipher_key)
def decrypt(text):
   return cf.hill_decrypt(text, hillcipher_inv_key)

client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
host = "127.0.0.1"
port = 12345
client_socket.connect((host, port))
print("Connected to the server")

while True:
   message = input("You: ")
   message = encrypt(message)
   print(f"Encrypted message sent to server: {message}\n")
   client_socket.send(message.encode())
   if decrypt(message).lower() == "exit":
       break
   msg = client_socket.recv(1024).decode()
   print(f"Encrypted message from server: {msg}")
   data = decrypt(msg)
   print(f"Decrypted message from server: {data}\n")

client_socket.close()
```

Output:

```
(lab) ak@DevLab:~/Desktop/Data/College/Semester6/Assignment/Crypto Lab/Ex1b$ python3 ./server.py
Server listening on 127.0.0.1:12345
Connected to ('127.0.0.1', 42772)
Encrypted message from client: HIOZFBYQSUAYFETCOZESONLPDSWOZZ
Decrypted message from client: HELLOFIAMUSINGHILLCIPHERNOWUUX

You: Okay, but these ciphers doesn't help with spaces? and symbols also!!
Encrypted message sent to client: UAUQSPNFAVOUSSRNHIBUZYOUNAAVTLHKDHXATESYWANNLSEETUJIHDSCQK

Encrypted message from client: GQRJACOGOYDPXKEZVXVELTASZQXP
Decrypted message from client: YESFISAWUCONVERTEDTOUBUGXUCX

You: okieee then, it's enough demonstation byee
Encrypted message sent to client: UAKKYCAVZVNYFMOEDSASEDWQDPHBFROIQFGQDT

Encrypted message from client:
Decrypted message from client:
```

```
(lab) ak@DevLab:~/Desktop/Data/College/Semester6/Assignment/Crypto Lab/Ex1b$ python3 ./client.py
Connected to the server
You: Hello, I am using Hill cipher now!!
Encrypted message sent to server: HIOZFBYQSUAYFETCOZESONLPDSWOZZ

Encrypted message from server: UAUQSPNFAVOUSSRNHIBUZYOUNAAVTLHKDHXATESYWANNLSEETUJIHDSCQK
Decrypted message from server: OKAYFBUTTHESECIPHERSDOESNATHELPWITHSPACESYANDSYMBOLSALSOUU

You: Yes, i saw ! converted to U (!->U)
Encrypted message sent to server: GQRJACOGOYDPXKEZVXVELTASZQXP

Encrypted message from server: UAKKYCAVZVNYFMOEDSASEDWQDPHBFROIQFGQDT
Decrypted message from server: OKIEEETHENFITASENOUGHDEMONSTATIONBYEEX

You: exit
(lab) ak@DevLab:~/Desktop/Data/College/Semester6/Assignment/Crypto Lab/Ex1b$
```